pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Rolling Windows and Volatility

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: one definition of volatility

The Desk Lead's Note

Two of our screens show the 10-year's volatility as 5.29 and 84.05 bp, from the same Treasury file, and neither says what it is. Give "marketdata" one definition: a window in rows, trailing, full windows only, the sample standard deviation of daily changes in bp, per day unless the name says annual. Tie it to Excel's "STDEV.S" filled down, prove no future row ever reaches today's number, and show me the 10-year over about one, three and twelve months as of 2 October. Describe how much it moved, with no view on what comes next.

The lesson reads only the Treasury file, and the exercises the synthetic spreads, invented for this course. There is no key cell and no live cell today.

```
Work folder pfi_course3_07_k4w7q2xr: 4 data files, Course 2's bondmath.py, marketdata.py as notebook 06 left it, 6 commits, tagged week1
.........................   [100%]
25 passed in 0.50s
```

Imports and the 10 Yr's Changes

```
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
ten = treasury['10 Yr']
ten_changes = md.change_bp(ten).dropna()
print(f'{len(ten):,} levels, {len(ten_changes):,} daily changes in bp')
2,940 levels, 2,939 daily changes in bp
```

Notebook 06 measured each day's move; today measures how much those moves vary.

7.1 A window in rows against a window in days

The note's first choice is what a window holds. Course 1's Treasury Yields case study used "rolling(20)", the last 20 rows, its picture "A 20-Row Window". "rolling('7D')" takes every row dated in the last 7 calendar days instead. ".count()" after ".rolling()" counts the values in each window.

Thirty Days, Every Date

```
days_30 = ten_changes.rolling('30D').count()
counts = days_30.loc['2015-02':].value_counts().sort_index()
share = counts[21] / counts.sum() * 100
```

"rolling(21)" holds 21 on every date.

The course counts rows: 21, 63 and 252, "about one month, three months, one year", conventions rather than calendar months.

The First Date

```
first = pd.DataFrame({"rolling('30D')": ten_changes.rolling('30D').mean(),
                      'rolling(21)': ten_changes.rolling(21).mean()})
first.head(3).round(2)
```

date	rolling('30D')	rolling(21)
2015-01-05	-8.00	NaN
2015-01-06	-7.50	NaN
2015-01-07	-5.33	NaN

The 30-day window gives a mean of one change on the first date; the 21-row window waits. pandas waits for a full window of rows, but gives a time window a number from its first row.

Excel side by side. Dates in A, the 10 Yr's changes from row 3.

Job	Python	Excel	Result
Rows in the last 30 calendar days, filled down	`ten_changes.rolling('30D').count()`	`=COUNTIFS($A$3:$A$2941,">"&A3-30,$A$3:$A$2941,"<="&A3)`	Equal on all 2,939 rows, 21 on the last

7.2 Full windows: trailing, right-aligned, min_periods

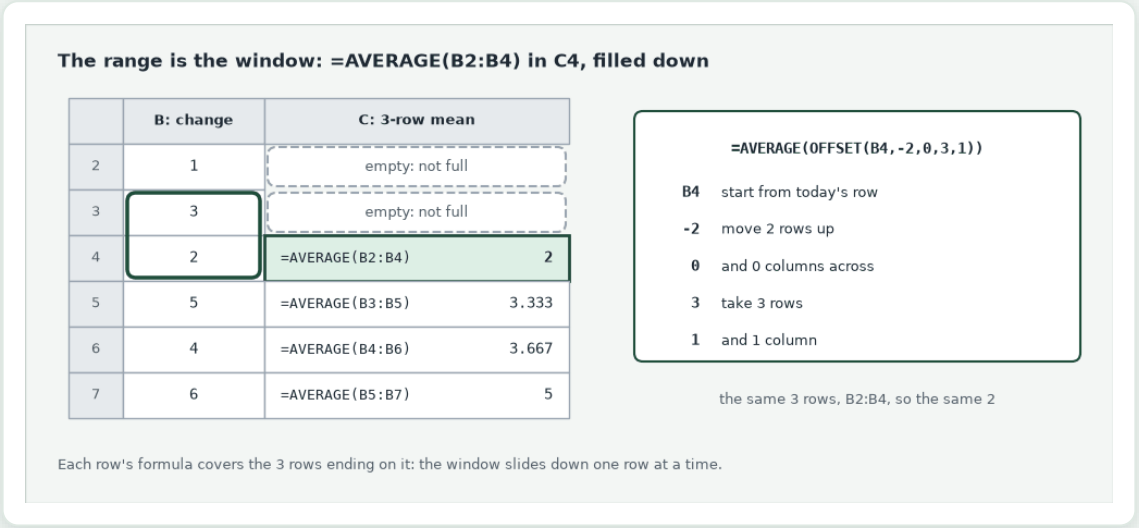
The 21-row window waited for 21 rows. Trailing means the window ending on t holds t and the rows before it. Right-aligned means pandas stores that window's value on t, as Course 1's picture showed. "min_periods" is how many values a window needs before it gives a number.

Six Changes, Two Rules

```
hand = pd.Series([1.0, 3.0, 2.0, 5.0, 4.0, 6.0])
means = pd.DataFrame({'change': hand,
                      'full windows': hand.rolling(3, min_periods=3).mean(),
                      'min_periods=1': hand.rolling(3, min_periods=1).mean()})
means.round(2)
```

	change	full windows	min_periods=1
0	1.0	NaN	1.00
1	3.0	NaN	2.00
2	2.0	2.00	2.00
3	5.0	3.33	3.33
4	4.0	3.67	3.67
5	6.0	5.00	5.00

By hand, the first full window is (1 + 3 + 2) / 3 = 2. The first two "min_periods=1" values are means of one and two changes, a different statistic under the same name.



The same windows in Excel, and 63 rows of the 10 Yr

Excel side by side. The six changes in B2:B7.

Job	Python	Excel	Result
3-row mean, C4 filled down	`hand.rolling(3, min_periods=3).mean()`	`=AVERAGE(B2:B4)`	2, 3.333, 3.667, 5, with C2 and C3 empty
The same window by offset	`the same`	`=AVERAGE(OFFSET(B4,-2,0,3,1))`	2, 3.333, 3.667, 5
A growing range, the changes in A1 to A3	`min_periods=1`	`=AVERAGE(A$1:A1) filled down`	1, 2, 2

63 Rows of the 10 Yr Against Excel

```
reference = pd.read_csv('course3_excel_reference.csv', parse_dates=['date'], index_col='date',
                        float_precision='round_trip')
mean63 = ten.rolling(63, min_periods=63).mean()
gap = (mean63.loc[reference.index] - reference['mean63_10y_pct']).abs().max()
print(f'{len(reference)} rows of 2026, largest gap {gap:.1e} percent')
190 rows of 2026, largest gap 1.8e-15 percent
```

Full windows only: "min_periods" equals the window, so the first window - 1 values are NaN, as Excel leaves the cells above the first full range empty.

Excel side by side. The whole file on one sheet: dates in A, the 2, 5, 10 and 30 Yr in B to E from row 2, and the 10 Yr's change "=(D3-D2)*100" in F3 filled down.

Job	Python	Excel	Result
63-row mean of the 10 Yr, L64 filled down	`ten.rolling(63, min_periods=63).mean()`	`=AVERAGE(D2:D64)`	Within 2.7e-15 percent on every row
The same by offset, M64	`the same`	`=AVERAGE(OFFSET(D64,-62,0,63,1))`	Within 2.7e-15 percent on every row

7.3 Volatility in bp: rolling_vol_bp

```
%%writefile -a marketdata.py
import math  # the annualising factor, from day 7
TRADING_DAYS_PER_YEAR = 252
def rolling_vol_bp(changes_bp: pd.Series, window: int = 63, annualise: bool = False) -> pd.Series:
    """Rolling volatility of daily changes in bp: the sample standard deviation over a trailing window.
    Units: changes in bp in, bp out (per day; per year when annualise=True).
    Convention: the window ending on date t holds t and the window - 1 rows before it (rows, not calendar
    days); min_periods = window, so the first window - 1 values are NaN; sample standard deviation (ddof=1,
    Excel STDEV.S). annualise=True multiplies by sqrt(252), a convention.
    Excel: =STDEV.S(F2:F64) filled down, and =STDEV.S(F2:F64)*SQRT(252)
    Raises ValueError if window is below 2, the changes have missing values (the first change is NaN, so pass
    change_bp(...).dropna()), or the index is not unique ascending dates.
    """
    _check_series(changes_bp, "changes_bp")
    if window < 2:
        raise ValueError(f"window must be at least 2 rows, not {window}")
    vol = changes_bp.rolling(window, min_periods=window).std(ddof=1)
    if annualise:
        vol = vol * math.sqrt(TRADING_DAYS_PER_YEAR)
    return vol
```

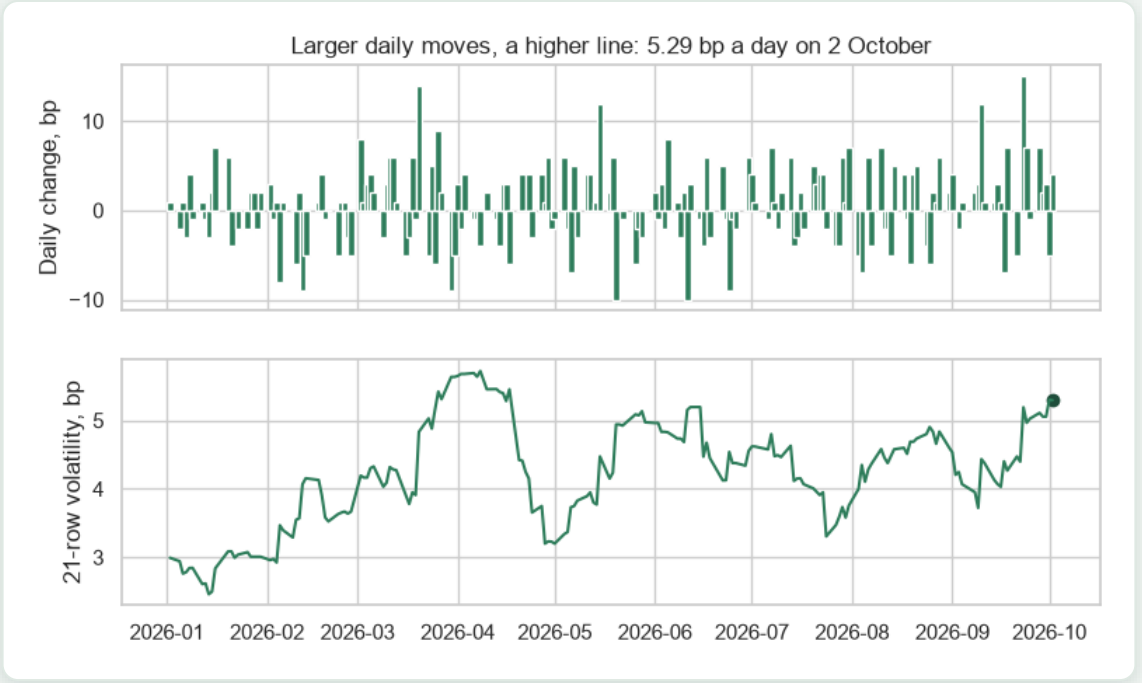
Now the statistic itself, over the changes and never the levels. Changes go in, in bp, and the result comes out in bp per day. The window counts rows, with "min_periods" equal to it, and the standard deviation is the sample one, "ddof=1", Excel's "STDEV.S". "annualise" is 7.5's subject. It refuses a window below 2 and, through notebook 04's "_check_series", a missing value or a bad index.

"math.sqrt" is the square root, as Course 1's "np.sqrt".

The note's 5.29, and every 2026 row against STDEV.S

```
importlib.reload(md)
vol21 = md.rolling_vol_bp(ten_changes, window=21)
print(f'{vol21.isna().sum()} NaN, the first value on {vol21.dropna().index[0]:%Y-%m-%d}')
print(f'2026-10-02: {vol21.iloc[-1]:.2f} bp a day')
20 NaN, the first value on 2015-02-03
2026-10-02: 5.29 bp a day
```

The note's first number, with a definition behind it.



Every 2026 Row Against STDEV.S

```
gap = (vol21.loc[reference.index] - reference['vol21_10y_bp']).abs().max()
print(f'{len(reference)} rows of 2026, largest gap {gap:.1e} bp')
print(reference['vol21_10y_bp_formula'].iloc[0])
190 rows of 2026, largest gap 2.4e-14 bp
=STDEV.S(F242:F262)
```

Python and Excel differ only in the last binary digits, far inside the course's 1e-9 bp. The reference sheet starts in December 2024, so its row numbers differ from the whole-file sheet below.

Excel side by side. The whole-file sheet of 7.2.

Job	Python	Excel	Result
21-row volatility, G23 filled down	`md.rolling_vol_bp(ten_changes, window=21)`	`=STDEV.S(F3:F23)`	Within 2.4e-14 bp on all 2,919 rows, the last 5.294651389216601
The same by offset, N23	`the same`	`=STDEV.S(OFFSET(F23,-20,0,21,1))`	Within 2.4e-14 bp

Two tests, and a forgotten dropna

```
%%writefile -a test_marketdata.py
def ten_year_changes() -> pd.Series:
    """Daily changes of the 10 Yr in bp, the first (NaN) dropped."""
    return md.change_bp(read_treasury()["10 Yr"]).dropna()
def test_vol_matches_excel_reference():
    reference = read_reference()
    vol = md.rolling_vol_bp(ten_year_changes(), window=21)
    vol_annual = md.rolling_vol_bp(ten_year_changes(), window=21, annualise=True)
    for day, row in reference.iterrows():
        assert vol.loc[day] == pytest.approx(row["vol21_10y_bp"], abs=1e-9), day
        assert vol_annual.loc[day] == pytest.approx(row["vol21_10y_annual_bp"], abs=1e-8), day
def test_vol_min_periods_nan():
    vol = md.rolling_vol_bp(ten_year_changes(), window=21)
    assert vol.iloc[:20].isna().all()
    assert vol.iloc[20:].notna().all()
```

Forget .dropna()

```
md.rolling_vol_bp(md.change_bp(ten), window=21)
ValueError: changes_bp has 1 missing values; drop them first with .dropna()
```

```
.........................   [100%]
27 passed in 0.50s
```

The Excel test also checks the annual column, which 7.5 explains.

"rolling_vol_bp" is Excel's "STDEV.S" filled down, on every row of 2026: 5.29 bp a day for the 10 Yr on 2 October.

7.4 Sample against population

Excel has two standard deviations. The sample one divides by n - 1 and the population one by n, as in Course 1's NumPy Part 1. A window is a sample of the series, so the course uses the sample one.

	Divides by	pandas	NumPy	Excel
Sample	n - 1	`.std(), ddof=1 the default`	`np.std(x, ddof=1)`	`STDEV.S`
Population	n	`.std(ddof=0)`	`np.std(x), ddof=0 the default`	`STDEV.P`

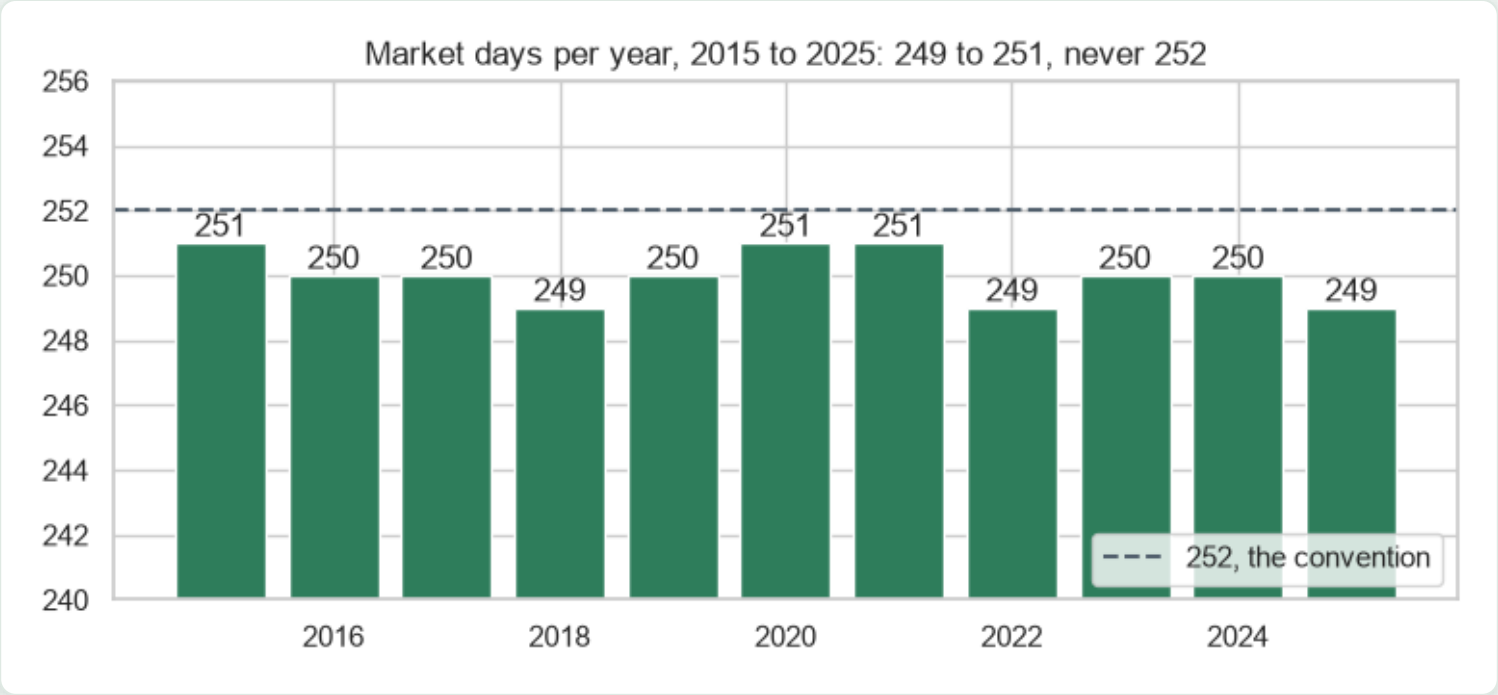
The Last 21 Changes, Both Ways

```
last_21 = ten_changes.iloc[-21:]
sample, population = last_21.std(ddof=1), last_21.std(ddof=0)
print(f'sample {sample:.6f}, population {population:.6f}, '
      f'np.std {np.std(last_21.to_numpy()):.6f} bp')
print(f'sample / population: {sample / population:.6f}')
sample 5.294651, population 5.167051, np.std 5.167051 bp
sample / population: 1.024695
```

Excel side by side.

Job	Python	Excel	Result
Population volatility, H23 filled down	`.rolling(21).std(ddof=0)`	`=STDEV.P(F3:F23)`	Within 2.4e-14 bp, the last 5.167050676973533
One value	`a window of 1`	`=STDEV.S(A1)`	#DIV/0!, which is why the window is at least 2

7.5 Annualising: the factor, labelled



Now the note's 84.05. Per year is per day times the square root of 252. Variances of independent days add, so the standard deviation grows with the square root of the number of days. 252 is a convention for the market days in a year, and other desks use others.

Excel side by side.

Job	Python	Excel	Result
Rows in 2015	`treasury.groupby(treasury.index.year).size()`	`=COUNTIFS($A$2:$A$2941,">="&DATE(2015,1,1),$A$2:$A$2941,"<"&DATE(2016,1,1))`	251, and equal for all 11 years

The note's 84.05, and the factor tested

The Note's 84.05

```
vol21_annual = md.rolling_vol_bp(ten_changes, window=21, annualise=True)
print(f'{vol21_annual.iloc[-1]:.2f} bp a year = {vol21.iloc[-1]:.2f} x {math.sqrt(252):.4f}')
gap = (vol21_annual.loc[reference.index] - reference['vol21_10y_annual_bp']).abs().max()
print(f'largest gap from Excel over 2026: {gap:.1e} bp')
84.05 bp a year = 5.29 x 15.8745
largest gap from Excel over 2026: 3.8e-13 bp
```

The two screens show one measure in two units.

Excel side by side. "=G23*SQRT(252)" in I23, filled down, is within 3.8e-13 bp on every row, the last 84.04998512789868.

The Factor, Tested

```
%%writefile -a test_marketdata.py
def test_vol_annualise_factor():
    changes = ten_year_changes()
    daily = md.rolling_vol_bp(changes, window=63)
    annual = md.rolling_vol_bp(changes, window=63, annualise=True)
    assert annual.iloc[-1] == pytest.approx(daily.iloc[-1] * math.sqrt(252), abs=1e-12)
```

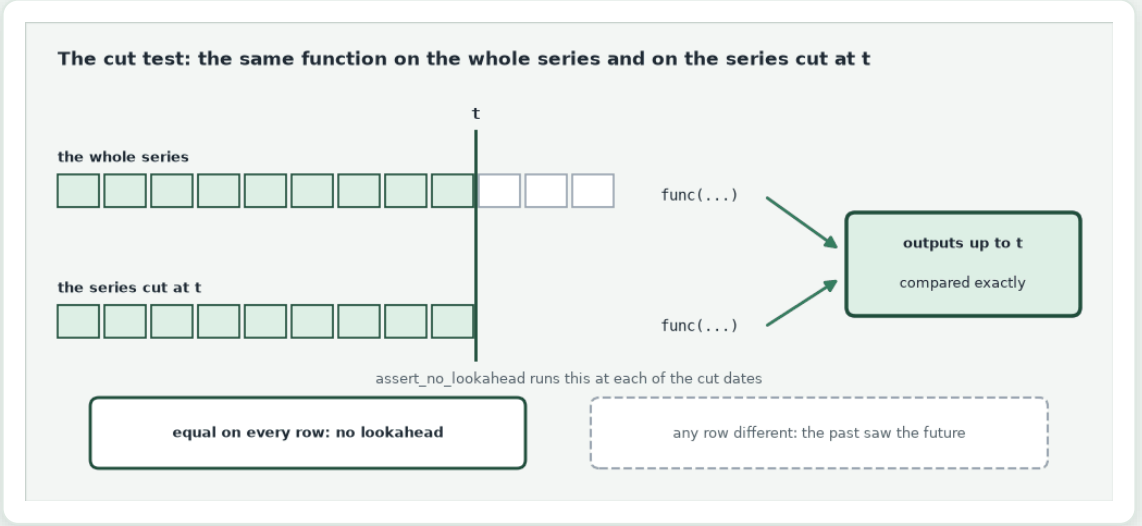
```
.........................   [100%]
28 passed in 0.50s
```

84.05 is 5.29 times the square root of 252, one measure in two units. The factor is a convention, so it lives in the name ("_annual") and the argument, never in silence.

7.6 Lookahead: assert_no_lookahead

Every window so far ends on its own date. Lookahead is a number for t built with rows after t. On a screen it looks like any other number. In a study of the past, it lets the past know the future.





A centred window reaches past t.

The rule: a function's output up to t is the same whether the series stops at t or runs on, compared exactly.

The helper and its test

```
%%writefile -a test_marketdata.py
def assert_no_lookahead(func, series, cut_dates):
    """Check that func uses no future data: its output up to each cut date is the same whether the series
    stops at that date or runs on past it (exact equality, no tolerance)."""
    full = func(series)
    for cut in cut_dates:
        cut = pd.Timestamp(cut)
        cut_result = func(series.loc[:cut])
        pd.testing.assert_series_equal(full.loc[:cut], cut_result, check_exact=True)
CUT_DATES = ["2025-04-17", "2026-06-30", "2026-09-30"]
def test_vol_no_lookahead():
    assert_no_lookahead(lambda s: md.rolling_vol_bp(s, window=63), ten_year_changes(), CUT_DATES)
```

"assert_series_equal(check_exact=True)" is notebook 05's, and "lambda" Course 1's one-line function. The cut dates are the Thursday before Good Friday 2025, a month end, and the file's last week.

```
.............................   [100%]
29 passed in 0.50s
```

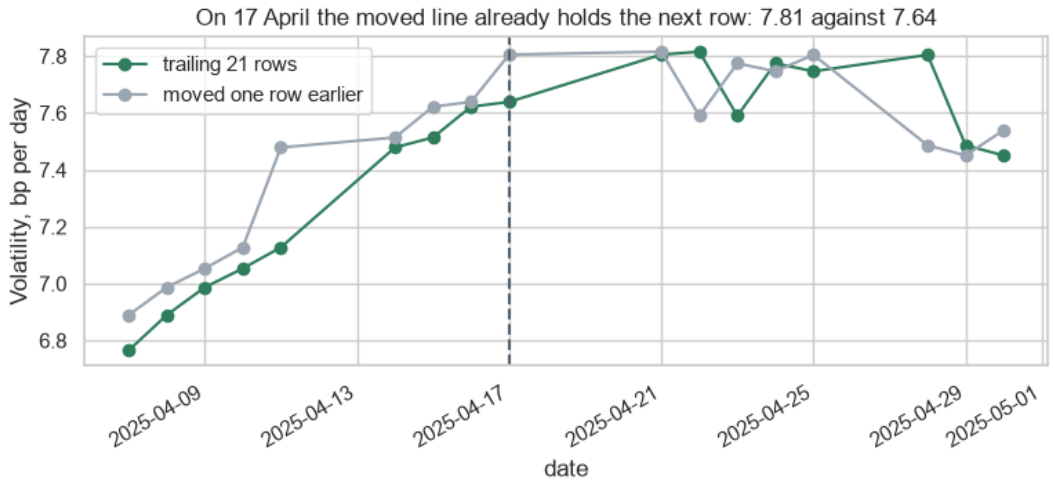
A value moved one row earlier

Q. A screen lines each day's volatility up with the next day's change. Does the check notice?

```
from test_marketdata import CUT_DATES
def vol_lined_up_with_next_day(changes):
    return md.rolling_vol_bp(changes, window=21).shift(-1)
```

```
full = vol_lined_up_with_next_day(ten_changes)
for cut in CUT_DATES:
    stopped = vol_lined_up_with_next_day(ten_changes.loc[:cut])
    assert full.loc[:cut].equals(stopped), f'values up to {cut} change when later rows arrive'
AssertionError: values up to 2025-04-17 change when later rows arrive
```

".equals" is Course 1's: the same values on the same dates, every one. It fails on the first cut date.



A test that knows nothing about how a function was written catches a value that saw the future. From today, every window function in the course passes it.

7.7 Yesterday's file, and back: git restore --source

The module is done for the day.

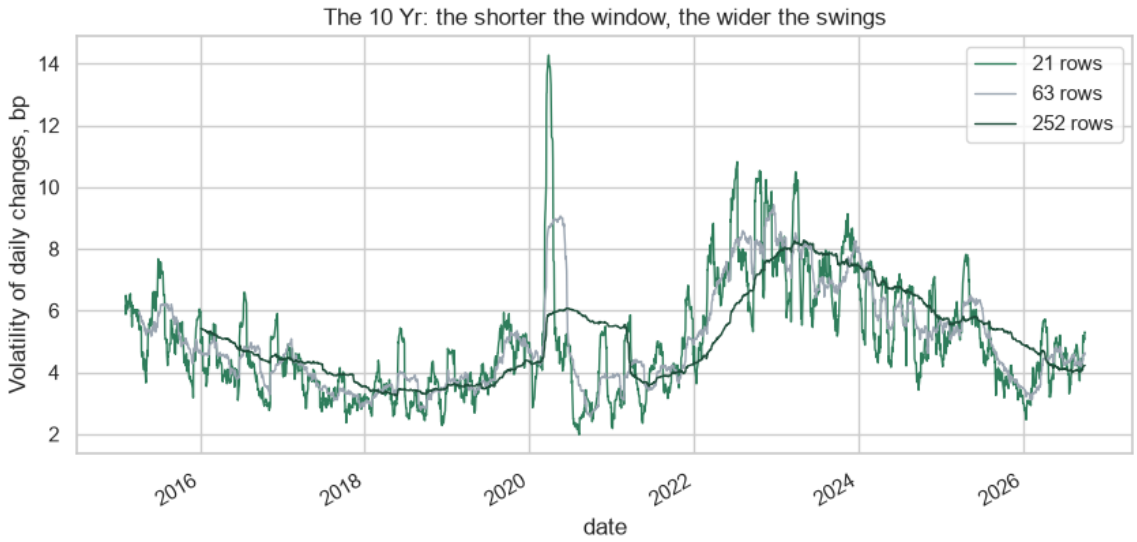
Read the Diff, Commit

```
!git -C "{work}" diff --stat
!git -C "{work}" add marketdata.py test_marketdata.py
!git -C "{work}" commit -q -m "Notebook 07: rolling_vol_bp and assert_no_lookahead"
!git -C "{work}" log --oneline -3
 marketdata.py   | 28 ++++++++++++++++++++++++++
 test_marketdata.py | 48 ++++++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 76 insertions(+)
85a1fa6 Notebook 07: rolling_vol_bp and assert_no_lookahead
1f5838a Notebook 06: change_bp, return_pct, and change_since_bp
ace289a Notebook 05: write_cache, read_cache, and cached_fred_series
```

"HEAD~1" is the commit before "HEAD", as Course 2's Duration and DV01 drew it in "Counting Back From HEAD". "--source=HEAD~1" tells "git restore" to take the file as it was in that commit.

7.8 21, 63 and 252 rows on one chart

The note's last ask.




```
for name in vols.columns:
    vol = vols[name].dropna()
    print(f'{name}: from {vol.index[0]:%Y-%m-%d}, low {vol.min():.2f} on {vol.idxmin():%Y-%m-%d}, '
          f'high {vol.max():.2f} on {vol.idxmax():%Y-%m-%d}, {vol.iloc[-1]:.2f} on 2026-10-02')
```

```
21 rows: from 2015-02-03, low 1.98 on 2020-08-10, high 14.27 on 2020-03-27, 5.29 on 2026-10-02
63 rows: from 2015-04-03, low 2.57 on 2020-09-29, high 9.43 on 2022-12-20, 4.63 on 2026-10-02
252 rows: from 2016-01-05, low 3.23 on 2018-05-21, high 8.26 on 2023-05-03, 4.22 on 2026-10-02
```

Three windows against Excel, as of 2 October

Excel side by side.

Job	Python	Excel	Result
63 rows, J65 filled down	`md.rolling_vol_bp(ten_changes, window=63)`	`=STDEV.S(F3:F65)`	Within 1.1e-14 bp, the last 4.628823961301718
252 rows, K254 filled down	`md.rolling_vol_bp(ten_changes, window=252)`	`=STDEV.S(F3:F254)`	Within 1.1e-14 bp, the last 4.222979359434287
The 21-row high	`vol21.max()`	`=MAX(G23:G2941)`	14.27202
Its date	`vol21.idxmax()`	`=XLOOKUP(MAX(G23:G2941),G23:G2941,A23:A2941)`	43917, 2020-03-27 as a date

Course 1's Treasury Yields case study read 5.34 over 20 rows. Today's 5.29 over 21 has a definition, an Excel tie and a no-lookahead test. Each line says how much the 10 Yr moved over its own window, and none says what comes next.

On 2 October 2026 the 10 Yr's daily changes had a volatility of 5.29 bp over 21 rows, 4.63 over 63 and 4.22 over 252. The note is answered.

Bring it home, and what you can now do

Copy the work folder's two files into your own "my_bondmath" folder, test them there, and commit. The copy replaces your two files, so anything of your own must be in the work folder first. The tests read the Treasury file and Excel's reference file, in your folder since notebooks 04 and 06. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_07_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 07: rolling_vol_bp and assert_no_lookahead"
git restore --source=HEAD~1 marketdata.py
git status --short
git restore marketdata.py
```

pytest shows 29 passed, and "status" shows " M marketdata.py" until the last line brings today's file back. In Colab, download the two files from the work folder under "/tmp" instead.

You Can Now

Tell a window in rows from a window in calendar days, and say why the course counts rows.

Keep to full windows with "min_periods", as Excel leaves the cells above the first full range empty.

Measure volatility with "rolling_vol_bp", tied to Excel's "STDEV.S" filled down.

Choose between the sample and population standard deviation, and name "ddof".

Annualise with the square root of 252, and label it.

Prove a window function uses no future data with "assert_no_lookahead".

Put yesterday's file in the folder with "git restore --source=HEAD~1", and bring today's back.

Predict first: a 7-day window, two ratios, a factor

Predict 1

Over the 10 Yr's changes, how many rows does each "rolling('7D')" window hold?

A) 5 on every date: a week has five market days

B) 4 or 5

C) anything from 2 to 7

```
days_7 = ten_changes.rolling('7D').count()
days_7.loc['2015-02':].value_counts()
```

Predict 2

At 21 rows the sample figure is 1.024695 times the population one. At 252 rows the gap is:

A) about the same, 2.5 percent

B) about 0.2 percent

C) about 25 percent

```
for n in [21, 252]:
    changes_n = ten_changes.iloc[-n:]
    print(f'{n} rows: {changes_n.std(ddof=1) / changes_n.std(ddof=0):.6f}')
```

Predict 3

Another desk annualises the same 5.29 bp a day with 250 days in place of 252. How many bp per year lower is its figure than 84.05? Write down a number.

```
for days in [252, 250]:
    print(f'{days} days: {vol21.iloc[-1] * math.sqrt(days):.2f} bp a year')
```

Predict first: its own average, and a restored file

Predict 4

Each value is a trailing 21-row volatility, nothing shifted, then divided by the average of the whole column, to show it against normal. The same check:

A) passes: every window ends on its own date

B) fails on the first cut date

C) fails only on the last cut date

```
def vol_against_its_average(changes):
    vol = md.rolling_vol_bp(changes, window=21)
    return vol / vol.mean()
full = vol_against_its_average(ten_changes)
for cut in CUT_DATES:
    stopped = vol_against_its_average(ten_changes.loc[:cut])
    assert full.loc[:cut].equals(stopped), f'values up to {cut} change when later rows arrive'
```

Predict 5

"git restore --source=HEAD~1 marketdata.py" puts notebook 06's "marketdata.py" in the folder. Then "git log --oneline -1" shows:

A) Notebook 06's commit: the history stepped back

B) today's commit, unchanged

C) a new commit holding the old file

```
!git -C "{work}" restore --source=HEAD~1 marketdata.py
!git -C "{work}" status --short
!git -C "{work}" log --oneline -1
print('def rolling_vol_bp' in (work / 'marketdata.py').read_text())
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: "center=True": values up to 2025-04-17 change when later rows arrive.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
What 5.29 and 84.05 are	One measure, the 21-row sample volatility of the 10 Yr's daily changes, per day and times the square root of 252
One definition in marketdata	rolling_vol_bp, in rows, trailing, full windows, ddof=1, per day unless annual
Tied to Excel	STDEV.S filled down, 190 of 190 rows of 2026, 29 passed
No future row	assert_no_lookahead on three cut dates
One, three and twelve months	5.29, 4.63 and 4.22 bp a day as of 2026-10-02

Be proud: few desk spreadsheets were ever asked whether a number saw the future, and yours now answers that in one line.

Next, Z-Scores and Percentile Ranks (notebook 08) uses the same windows to say where today sits in its own history.